Minor Project

AWS IAM — Roles, Policies & Permission Best Practices

Step-by-step work document: users, groups, custom policies, roles, MFA, and permission validation

1. Project Objective

Implement secure IAM access for ABC Technologies, covering three teams — Developers, DevOps Engineers, and Auditors — using IAM users, groups, custom and managed policies, an EC2 role, MFA, and forced password reset, then validate that the permissions actually behave as designed.

Architecture

AWS Account

Developers Group → Custom S3 Policy → developer1, developer2

DevOps Group → EC2/Admin Operations Policy → devops1, devops2

Auditors Group → ReadOnlyAccess → auditor1, auditor2

EC2 IAM Role → AmazonS3ReadOnlyAccess

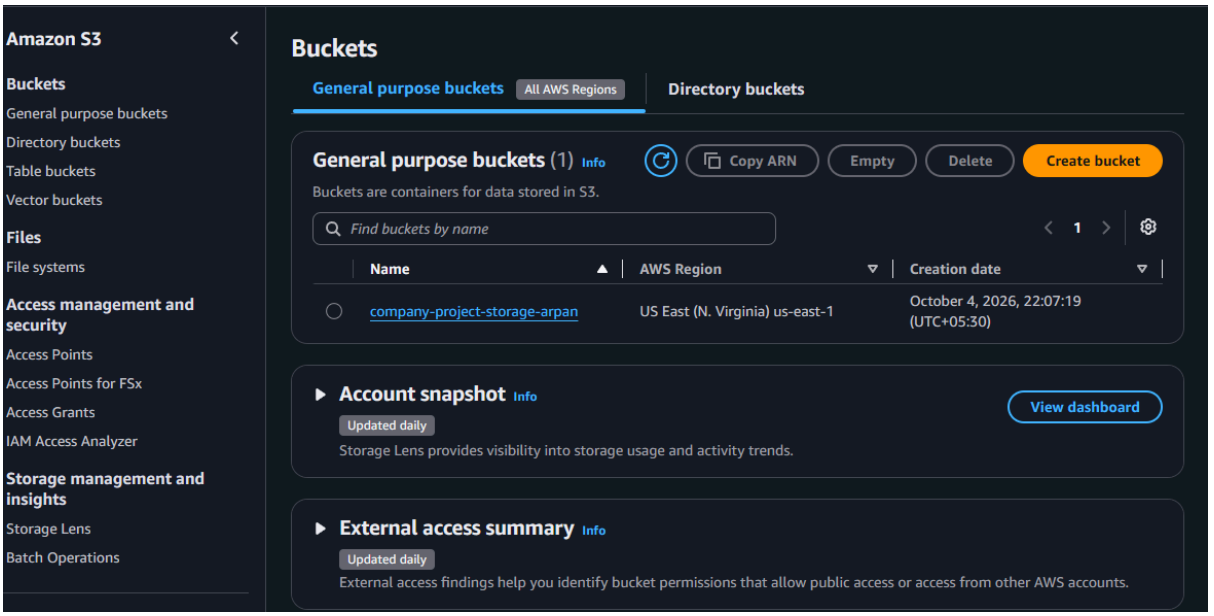
Configuration Summary

Item	Value
IAM users	developer1, developer2, devops1, devops2, auditor1, auditor2
IAM groups	Developers, DevOps, Auditors
S3 bucket	company-project-storage (created as company-project-storage-arpan, since the base name was taken)
Developers policy	DevelopersS3Policy (custom): s3:ListBucket, s3:GetObject, s3:PutObject on the project bucket only
DevOps policy	DevOpsOperationsPolicy (custom): EC2 lifecycle actions, CloudWatch read, read-only VPC/IAM role visibility
Auditors policy	ReadOnlyAccess (AWS managed, job function category)
EC2 role	EC2-S3-Access-Role: AmazonS3ReadOnlyAccess, attached to an EC2 instance
MFA	Enabled for developer1 (authenticator app)
Forced password reset	Enabled for developer2 at user creation

2. Step 1: Create the S3 Bucket

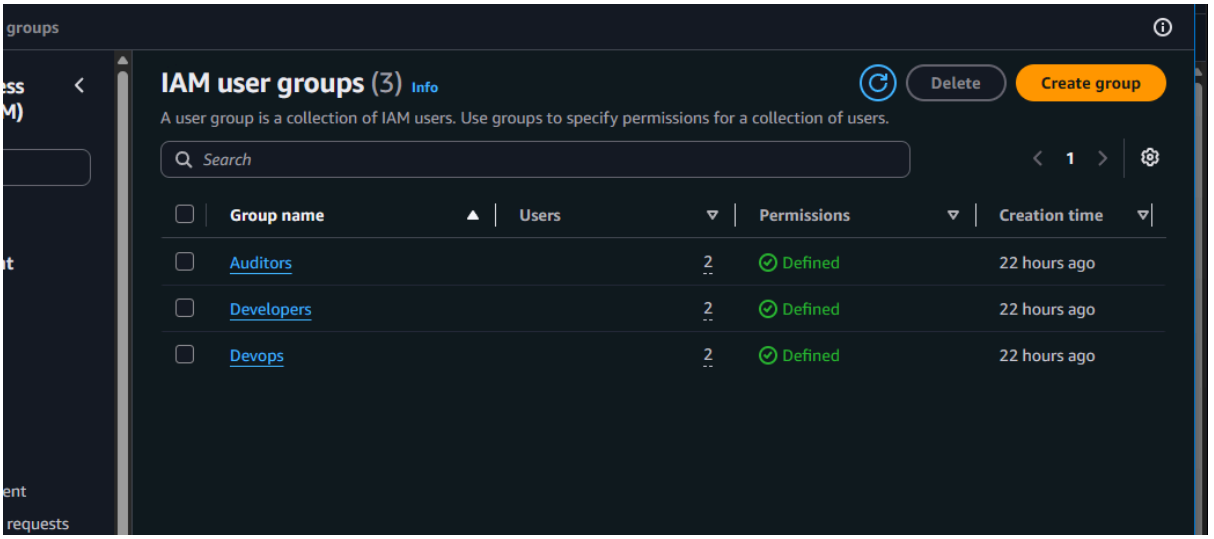
Create the bucket before the Developers policy, since the policy's JSON needs to reference the exact bucket name.

1. **S3 → Create bucket**.
2. Name: `company-project-storage`. Bucket names are globally unique across all AWS accounts — if that name is taken, add a suffix (for example `company-project-storage-arpan`) and use that exact name consistently in every policy below.
3. Region: any, just note which one.
4. Leave **Block all public access** ON.
5. Create the bucket, then upload 2–3 small sample files.



3. Step 2: Create the IAM Groups

1. **IAM → User groups → Create group**.
2. Create three groups, one at a time: `Developers`, `DevOps`, `Auditors`.
3. Don't attach policies yet — the custom policies are created in Steps 4–5 and attached afterward.



4. Step 3: Create the 6 IAM Users

The console only creates one user at a time — there is no bulk-create option. Repeat **IAM → Users → Create user** six times.

User details screen

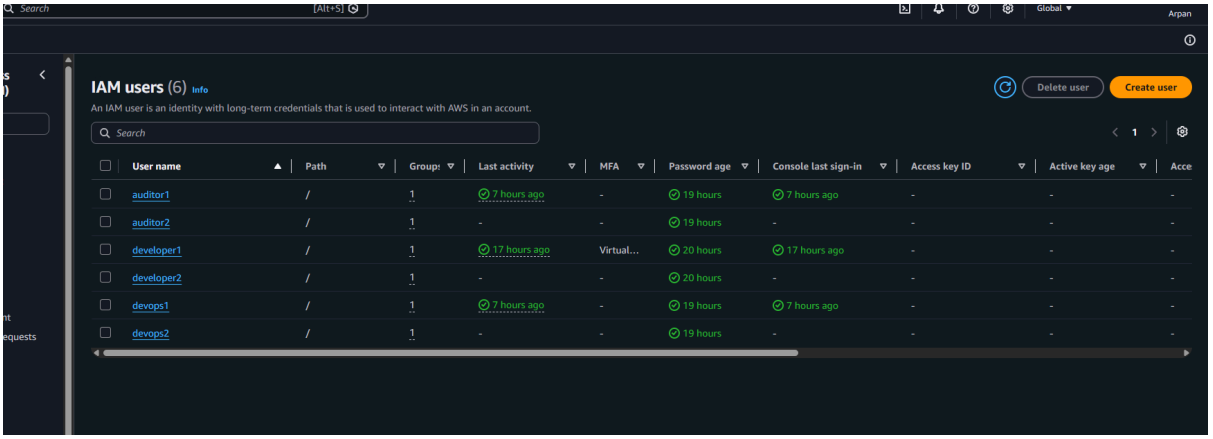
1. Username: e.g. `developer1`.
2. Tick **Provide user access to the AWS Management Console**, so the user can log in later for permission testing.
3. Choose **Custom password** and set one (write it down).
4. Leave **"Users must create a new password at next sign-in"** unticked, except for `developer2` (see below).

Permissions screen

1. Choose **Add user to group**.
2. Tick the matching group for this user.

Skip the Tags screen — click Next without adding anything — and click Create user.

Username	Group	Notes
developer1	Developers	Console access; MFA added in Step 8
developer2	Developers	Console access; tick "require password reset" at creation
devops1	DevOps	Console access
devops2	DevOps	Console access
auditor1	Auditors	Console access
auditor2	Auditors	Console access



5. Step 4: Create the Developers Policy

1. **IAM → Policies → Create policy → JSON tab**. Paste the policy below, replacing the bucket name if yours differs.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket",
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": [
        "arn:aws:s3:::company-project-storage-arpan",
        "arn:aws:s3:::company-project-storage-arpan/*"
      ]
    }
  ]
}
```

1. Name it `DevelopersS3Policy` and create it.
2. **IAM → User groups → Developers → Permissions tab → Add permissions → Attach policies**, search `DevelopersS3Policy`, attach it.

6. Step 5: Create the DevOps Policy

1. **IAM → Policies → Create policy → JSON tab**.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:StartInstances",
        "ec2:StopInstances",
        "ec2:RebootInstances",
        "ec2:TerminateInstances",
        "ec2:RunInstances",
        "ec2:DescribeInstances",
        "cloudwatch:GetMetricData",
        "cloudwatch:ListMetrics",
        "cloudwatch:DescribeAlarms",
```

```
        "ec2:DescribeVpcs",
        "ec2:DescribeSubnets",
        "iam:ListRoles",
        "iam:GetRole"
      ],
      "Resource": "*"
    }
  ]
}
```

1. Name it `DevOpsOperationsPolicy` and create it.
2. **IAM → User groups → Devops → Permissions tab → Add permissions → Attach policies**, search `DevOpsOperationsPolicy`, attach it.

7. Step 6: Attach ReadOnlyAccess to Auditors

1. **IAM → User groups → Auditors → Permissions tab → Add permissions → Attach policies**.
2. Search `ReadOnlyAccess`.
1. Tick the policy named exactly **ReadOnlyAccess** (type: AWS managed) and attach it.

8. Step 7: Create the EC2 IAM Role

1. **IAM → Roles → Create role**.
2. Trusted entity type: **AWS service**. Use case: **EC2**.
3. Attach policy: **AmazonS3ReadOnlyAccess** (AWS managed).
4. Name: `EC2-S3-Access-Role`. Create role.

Attach it to an instance:

1. **EC2 → select instance → Actions → Security → Modify IAM role**, choose `EC2-S3-Access-Role`, update. Or launch a new t3.micro instance and select this role under Advanced details → IAM instance profile at launch time.

9. Step 8: Enable MFA for developer1

1. **IAM → Users → developer1 → Security credentials tab → Assign MFA device**.
2. Name the device, choose **Authenticator app**, click Next.
3. Scan the QR code with an authenticator app (Google Authenticator, Authy, Microsoft Authenticator).
4. Enter two consecutive codes from the app to confirm, then **Add MFA**.

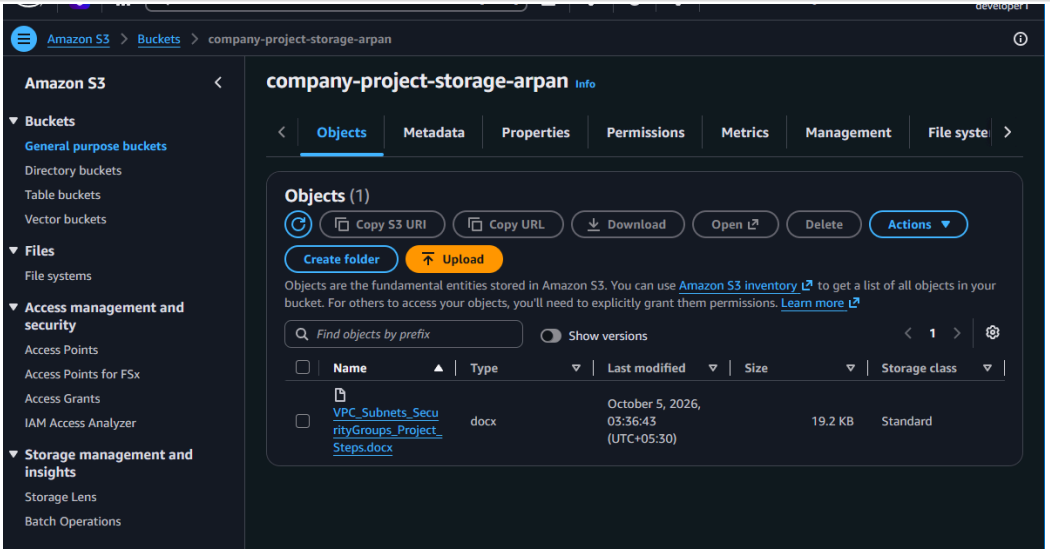
developer2's forced password reset was already configured at user creation (Step 3), satisfying that requirement without a separate step.

10. Step 9: Validate Permissions

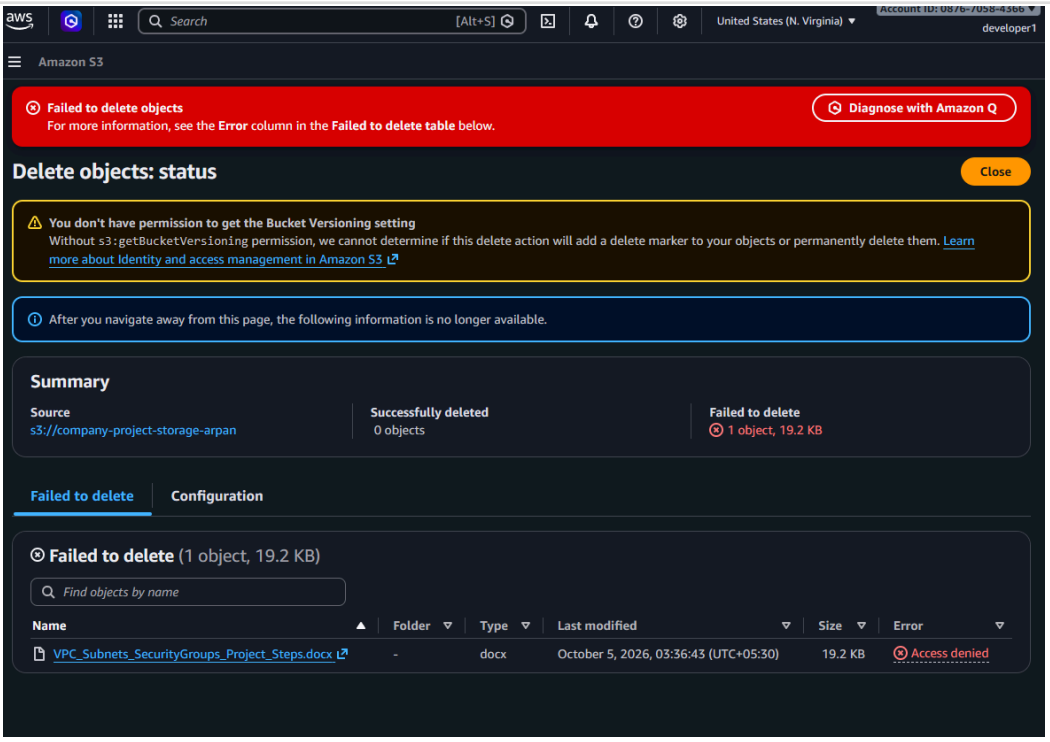
Log in as each user through the IAM sign-in URL (shown on the IAM dashboard, in the form https://<account-id>.signin.aws.amazon.com/console), using a separate incognito window per user so the admin session stays untouched.

User	Action	Expected	Result
developer1	Upload object	Success	Success
developer1	Delete object / bucket	Access Denied	Access Denied
auditor1	Create EC2	Access Denied	Access Denied
devops1	Terminate EC2	Success	Success

Test 1 & 2 — developer1: upload succeeds, delete is denied

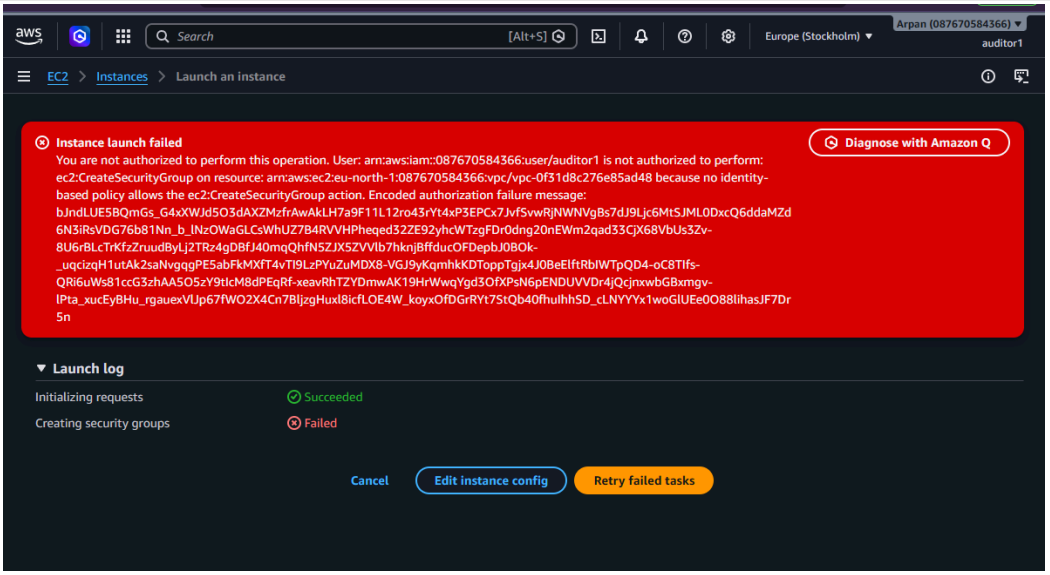


developer1 — object uploaded successfully to company-project-storage-arpan



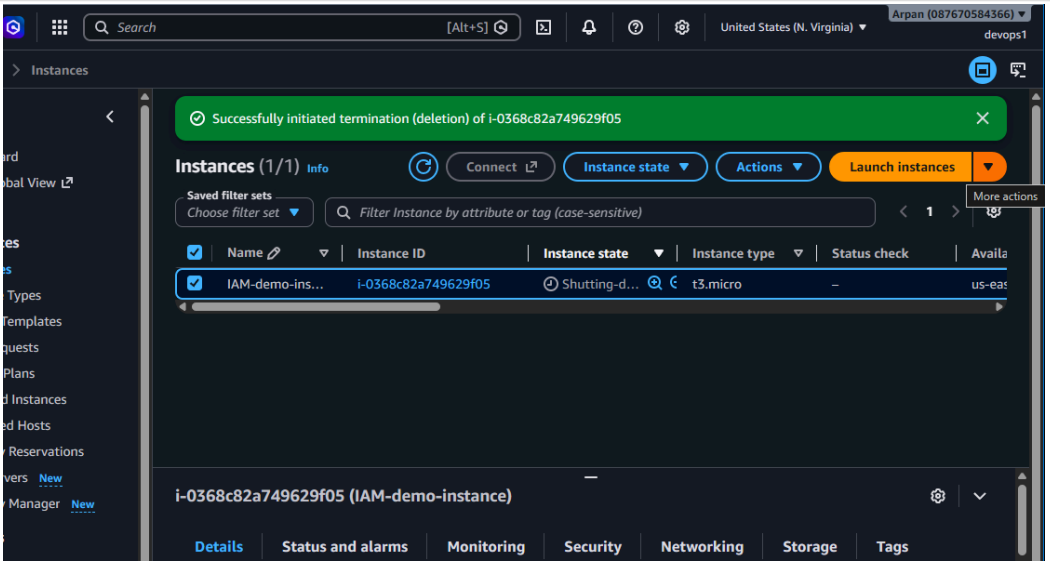
developer1 — delete attempt result: "Access denied"

Test 3 — auditor1: create EC2 is denied



auditor1 — instance launch failed: not authorized to perform ec2:CreateSecurityGroup

Test 4 — devops1: terminate EC2 succeeds



devops1 — "Successfully initiated termination (deletion)" of the instance

11. Deliverables Checklist

Deliverable	Captured in	Status
Screenshot of IAM Users	Step 3	Done
IAM Groups	Step 2, 4–6	Done
Custom Policies	Step 4–5	Done
IAM Role	Step 7	Done
S3 Bucket	Step 1	Done
EC2 with attached IAM Role	Step 7	Done
MFA enabled	Step 8	Done

Deliverable	Captured in	Status
Permission test results	Step 9	Done
Project documentation with learning outcomes	This document	Done

12. Troubleshooting

Problem	Fix
Policy attached to a group, but the user still can't access the resource	Check the policy's Resource ARN matches the actual bucket/resource name exactly — a bucket name suffix added because the base name was taken is a common silent mismatch.
Searching "ReadOnlyAccess" in Attach policies returns hundreds of results, or none at all	Use the Filter by Type dropdown and select Job function — this is a short list that includes the exact ReadOnlyAccess policy, bypassing an unreliable text search.
"You don't have permissions to list buckets" on the general S3 page	Expected for a user scoped to one bucket via s3:ListBucket, which is different from the account-wide s3:ListAllMyBuckets the overview page needs. Navigate directly to the bucket's URL instead.
Can't select a bucket to delete it (same list-page problem)	Test object-level delete from inside the bucket instead — it proves the same missing delete permission.
devops1 can't see any EC2 instances	Check the region selector in the top right matches the region the instance was actually launched in — EC2 instance lists are region-scoped.

13. IAM Best Practices Applied

- Enable MFA for sensitive accounts
- Use IAM groups instead of attaching permissions to individual users directly
- Apply least privilege — grant only the specific actions each role needs
- Avoid using the root account for day-to-day operations
- Use IAM roles (not long-term access keys) for EC2-to-AWS-service access
- Rotate credentials regularly
- Monitor IAM activity with CloudTrail
- Remove unused users and permissions

14. Learning Outcomes

- Creating and organizing IAM users into groups by job function
- Writing custom least-privilege IAM policies in JSON

- Distinguishing between resource-scoped actions (e.g. s3:ListBucket) and account-wide actions (e.g. s3:ListAllMyBuckets) that look similar but behave very differently
- Attaching AWS managed policies for standard access patterns (ReadOnlyAccess)
- Creating and attaching an IAM role for EC2-to-S3 access without long-term credentials
- Enabling MFA and forcing password resets as account-hardening measures
- Validating permissions empirically by testing as each user, rather than assuming a policy works as written